

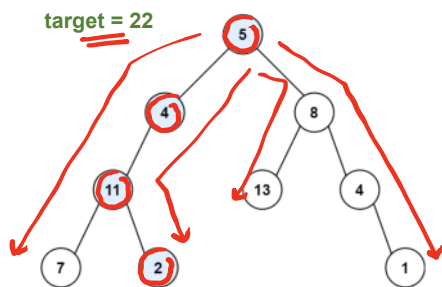
Path Sum I

Path Sum I

Given the **root** of a binary tree, and an integer **target**, design an algorithm to check if there exists a **root-to-leaf** path that sums up to the target or not.

$$5 + 4 + 11 + 2 = 22$$

o/p: true



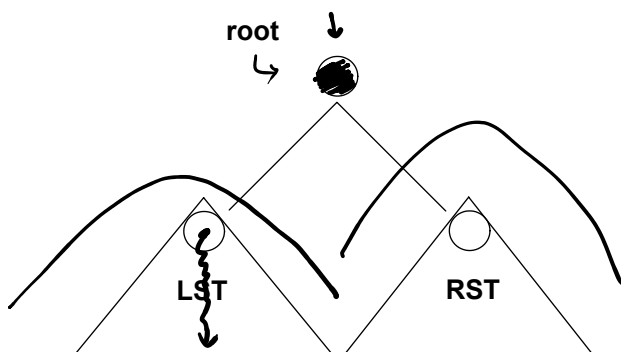
$$t = 29 \quad \text{o/p: false}$$

$$5 + 4 + 11 + 7 = 27$$

$$5 + 8 + 13 = 26$$

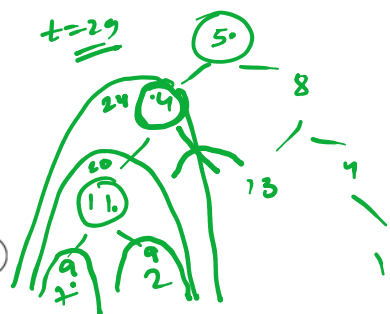
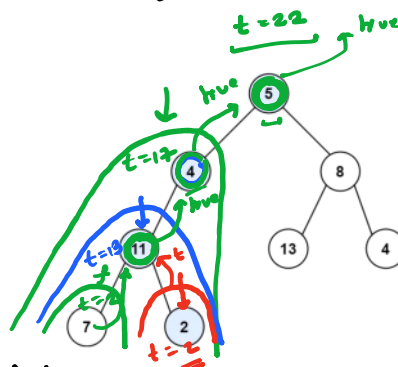
$$5 + 8 + 4 + 1 = 18$$

$f(\text{root})$:
check if there exists a root to leaf path whose sum is equal to target



ask your friend to find a root-to-leaf path in the LST whose sum is target
- root $\rightarrow$ val

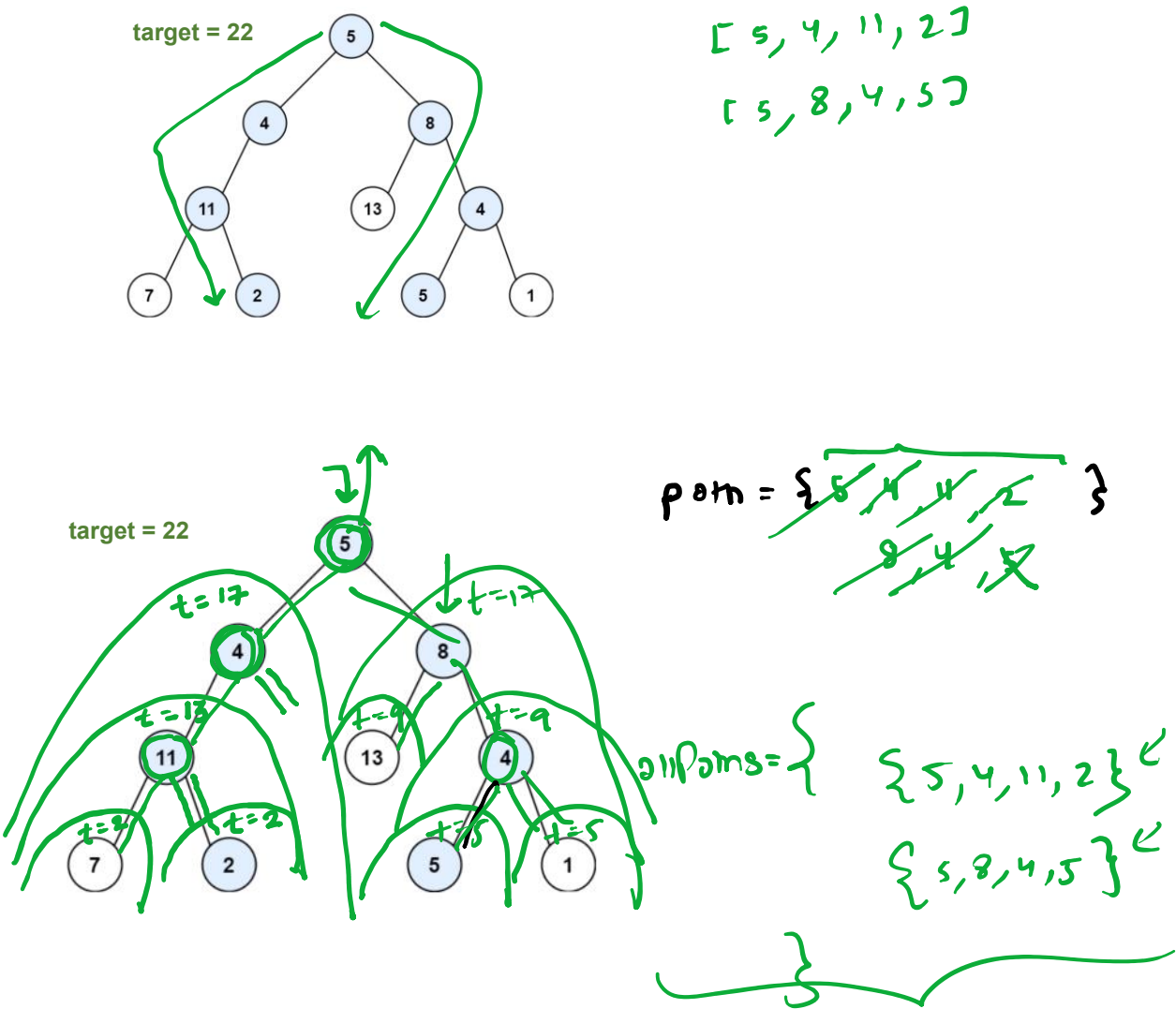
ask your friend to find root-to-leaf path in the RST whose sum is target
- root $\rightarrow$ val



Path Sum II

Path Sum II

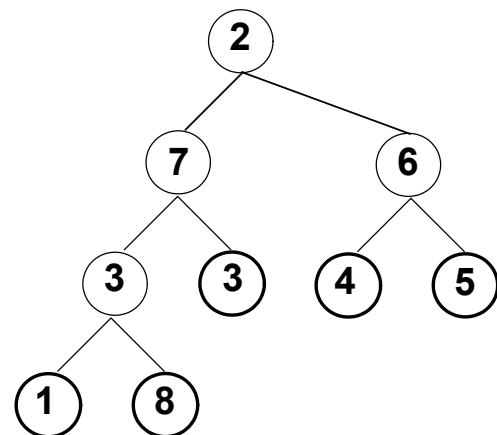
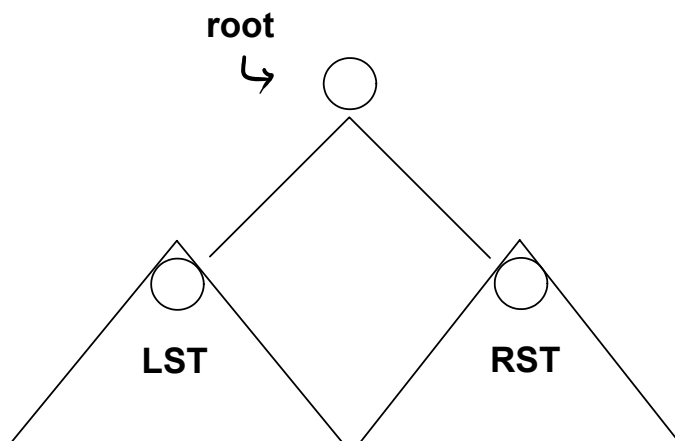
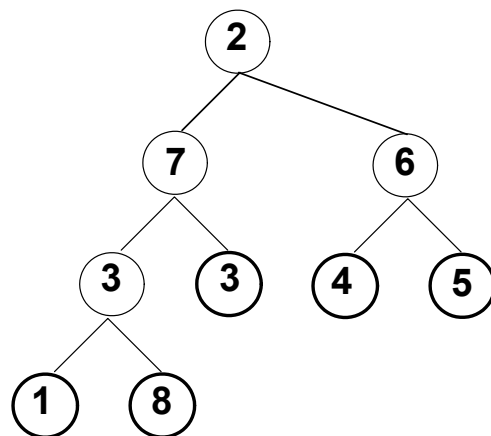
Given the **root** of a binary tree, and an integer **target**, generate all the **root-to-leaf** paths that exist in the binary tree and sum up to the target.



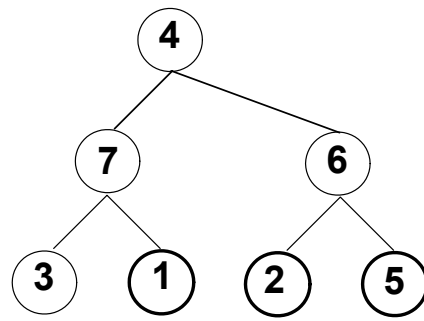
Root Path Sum

Given the **root** of a binary tree, and an integer **target**, design an algorithm to count the number of paths in the binary tree starting from the **root** node that sum up to the target.

target = 12

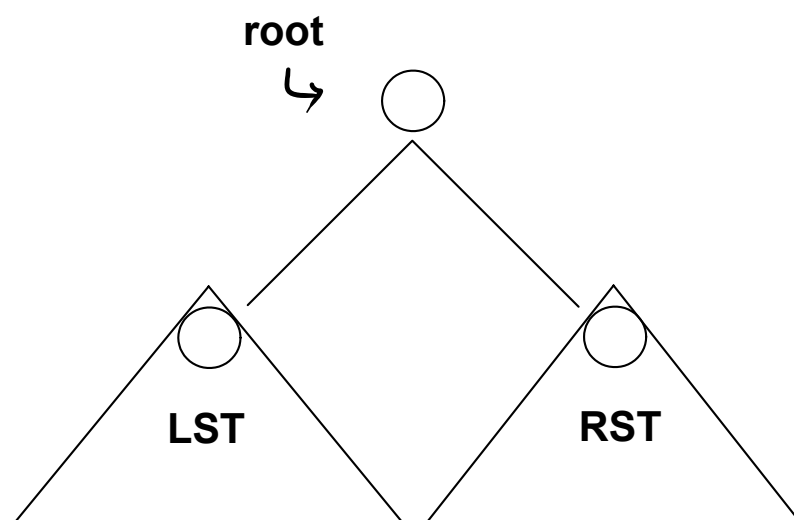
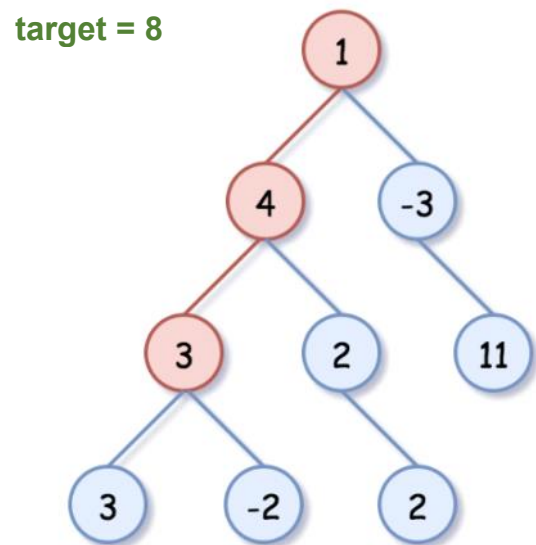
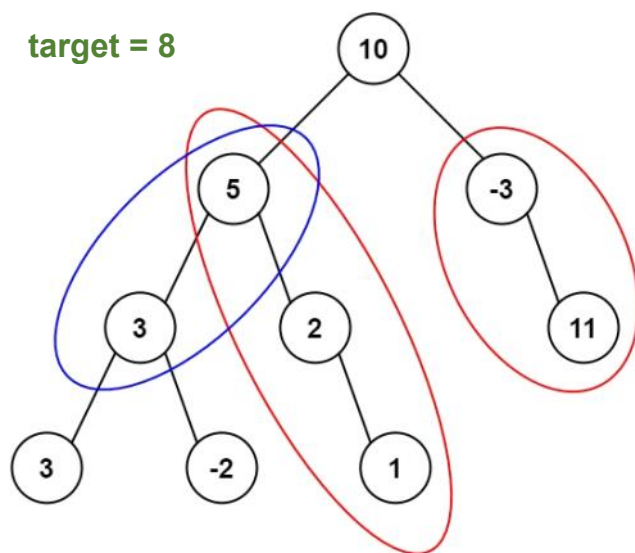


target = 4



Path Sum III

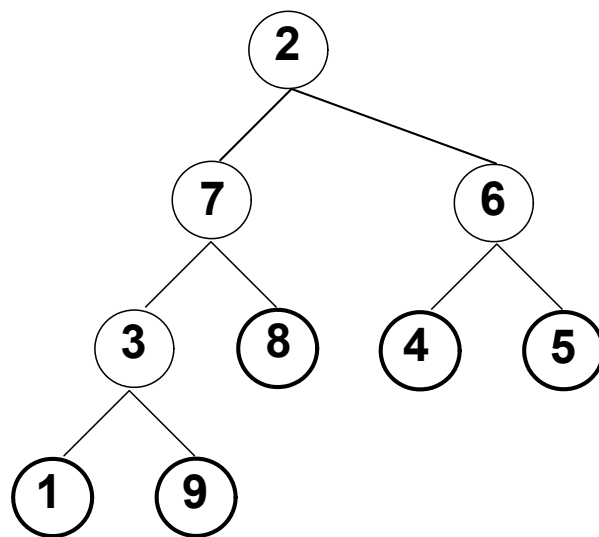
Given the **root** of a binary tree, and an integer **target**, design an algorithm to count the number of paths in the binary tree which sum up to the target.



[R][HW] Path to Node

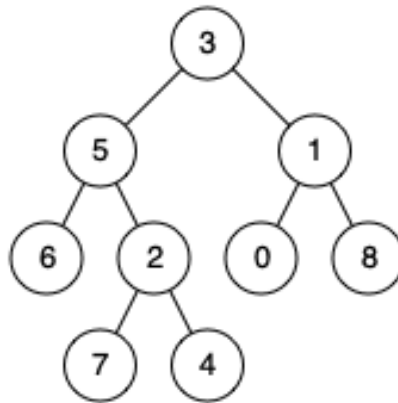
Path to Node

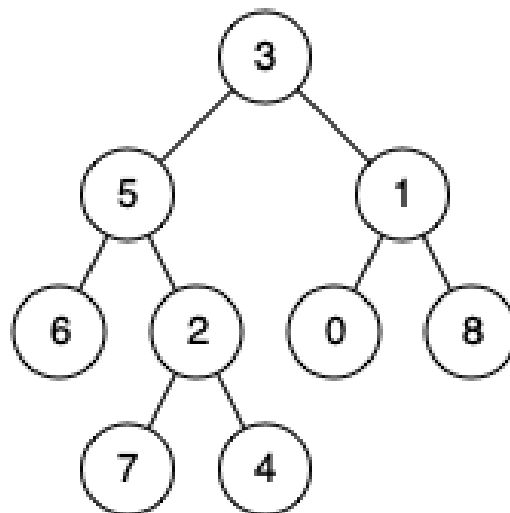
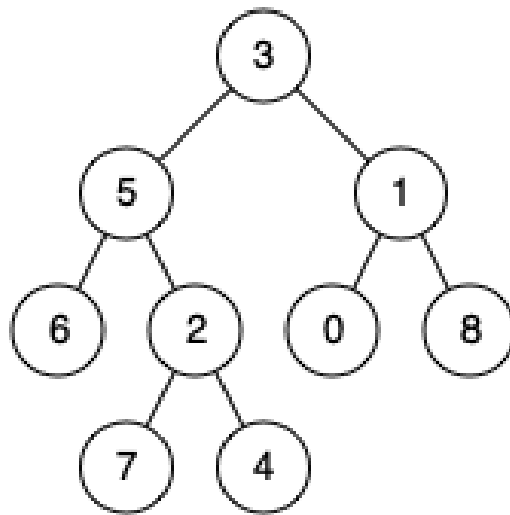
Given the **root** of a binary tree, and a **target** node, design an algorithm to generate the path from **root-to-target** .

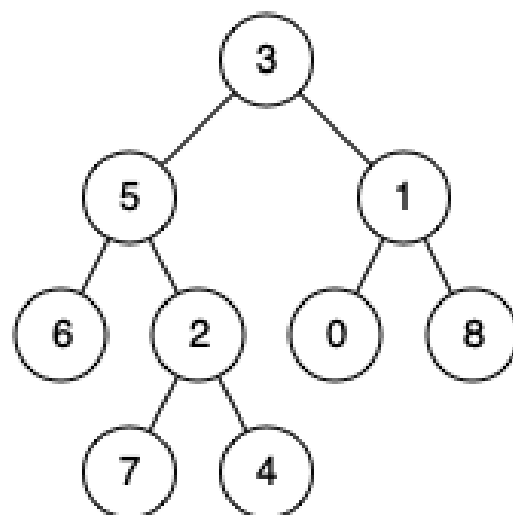
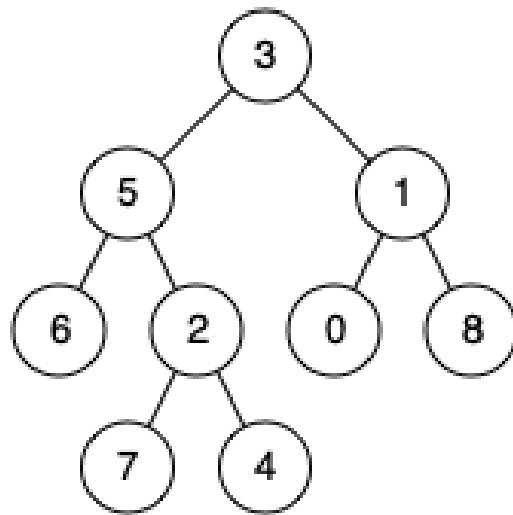


Lowest Common Ancestor of a Binary Tree

Given a binary tree, and two nodes **P** & **Q**, design an algorithm to find the lowest common ancestor (**LCA**) of the two given nodes in the tree. For a given tree **T**, we define the **LCA** between two nodes **P** & **Q** as the **lowest** node in the tree that has both P & Q as descendants (where we allow a node to be a descendant of itself.)



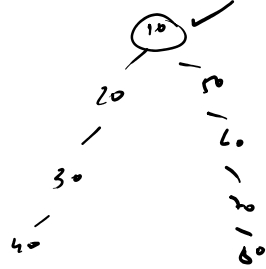
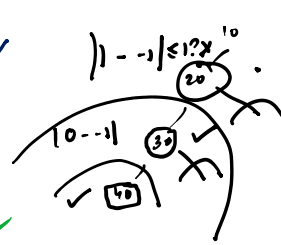
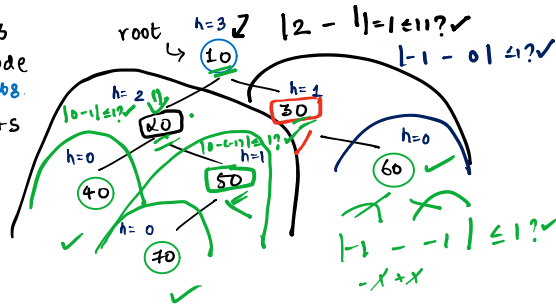




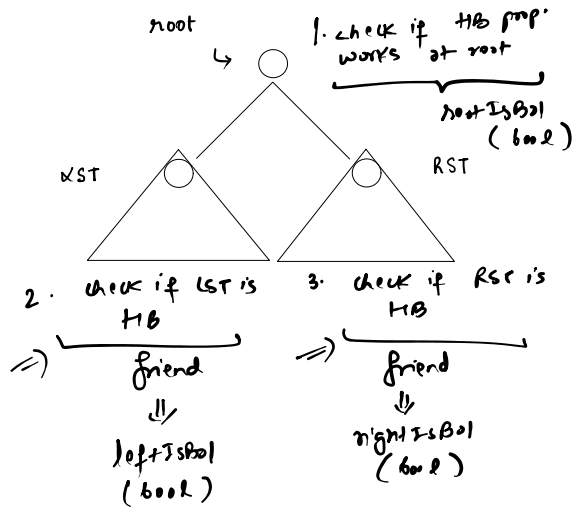
Height Balanced Binary Tree

↳ Check if a Binary Tree is height-balanced

We say that a binary-tree is **height-balanced** if for each node in the binary-tree the **abs difference** of the height of its LST and RST is ≤ 1 .

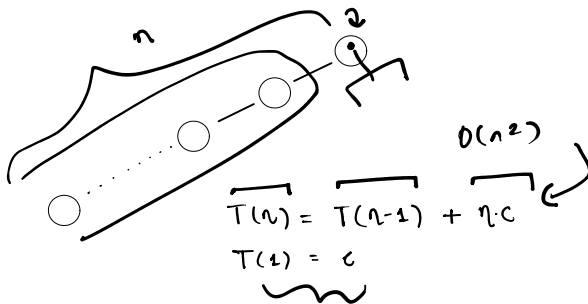


$f(\text{root})$: check if given tree is HB?

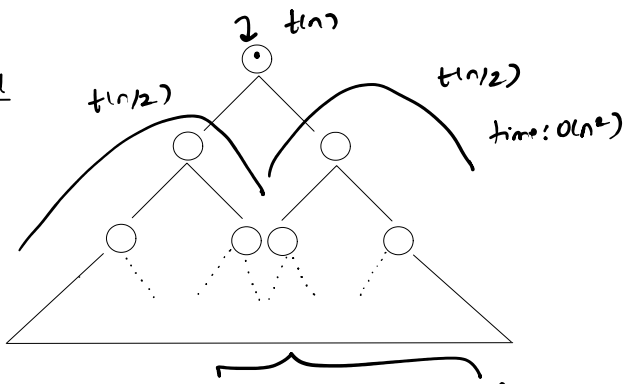


↳ Analysis of time-complexity using back-substitution

1. Skewed



2. Balanced



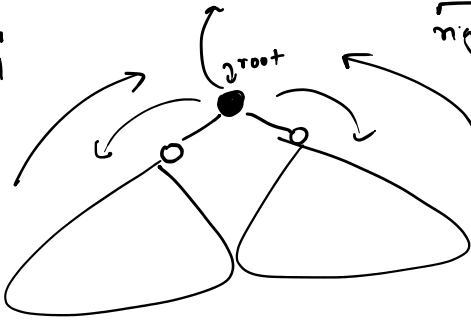
Recursion

$$T(n) = 2T(n/2) + n \cdot c$$

$$T(1) = c$$

(leftIsBal & rightIsBal & rootIsBal) $1 + \max(\text{leftHgt}, \text{rightHgt})$

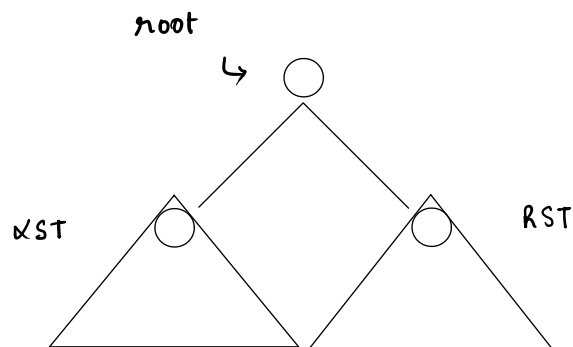
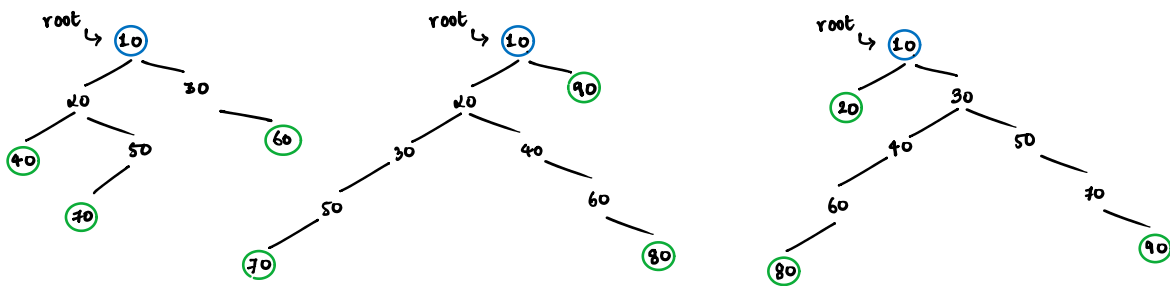
leftIsBal
(bool)
leftHgt
(int)



rightIsBal
(bool)
rightHgt
(int)

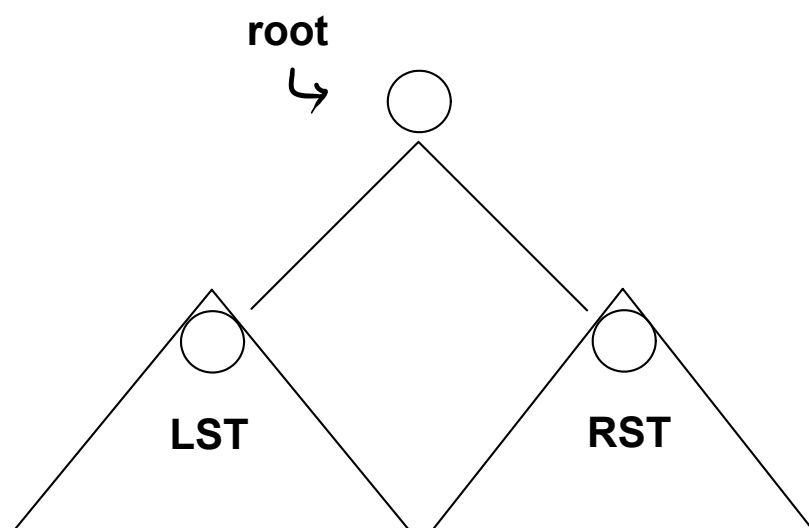
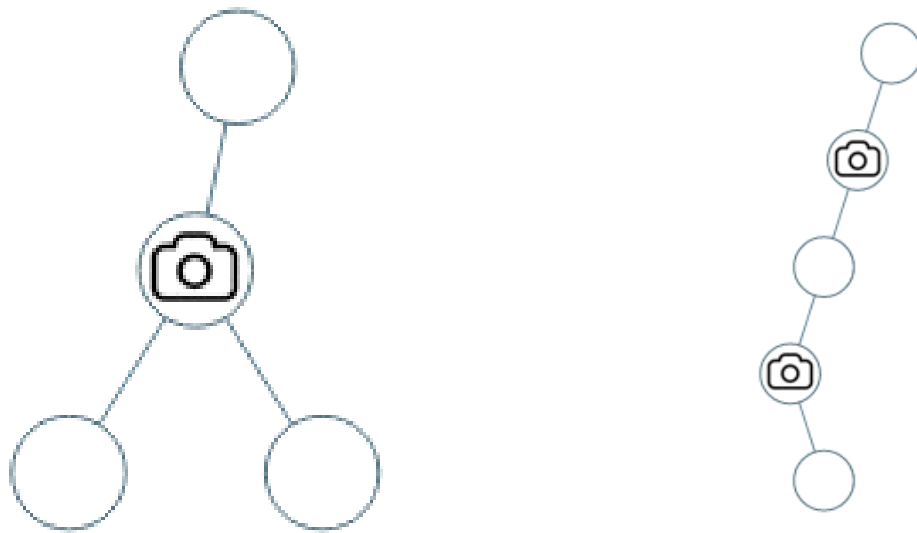
↳ Diameter of Binary Tree

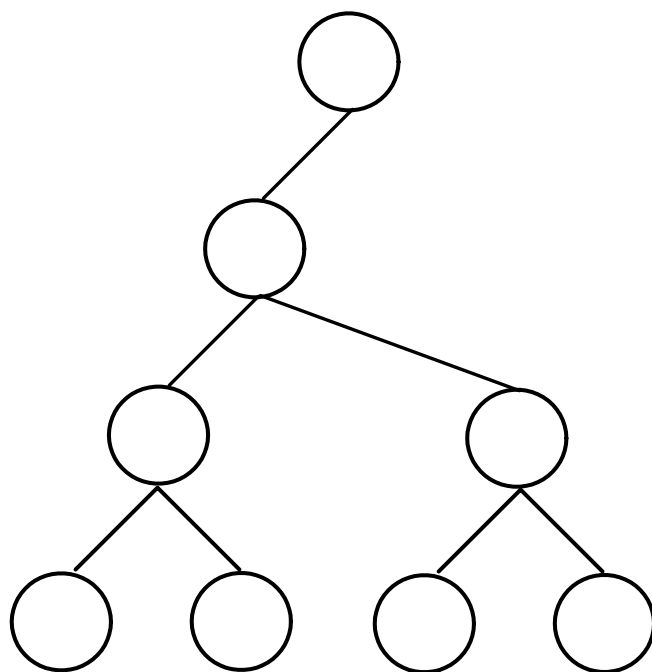
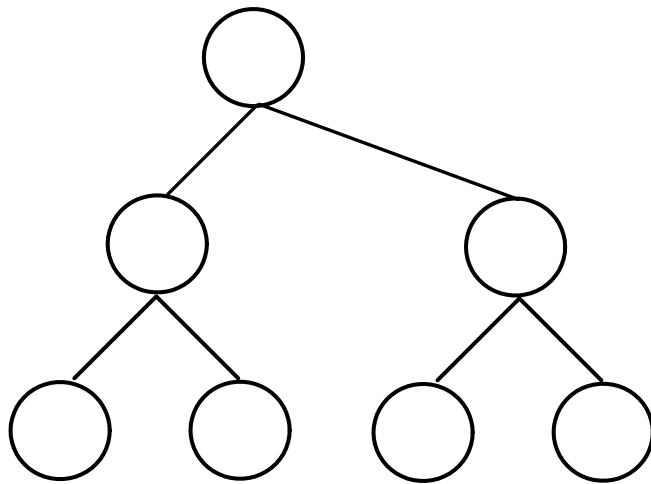
It is the length of the longest path b/w any two nodes in a binary tree.



Binary Tree Cameras

Given the **root** of a binary tree, find the **minimum no. of cameras** required to monitor all the nodes in the tree such that a camera installed a node can monitor itself, its parent and its immediate children.

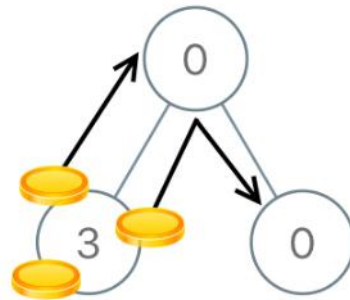
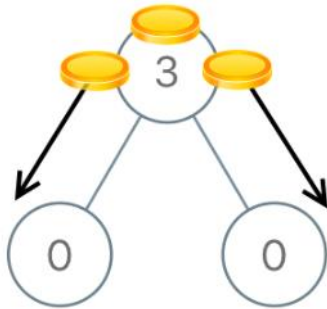




[HW] Distribute Coins in Binary Tree

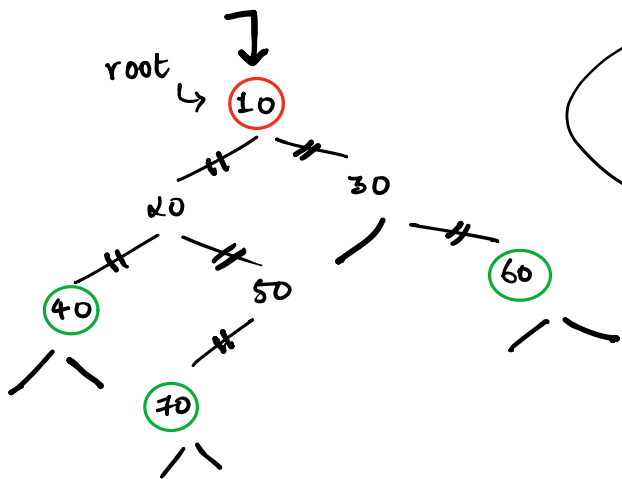
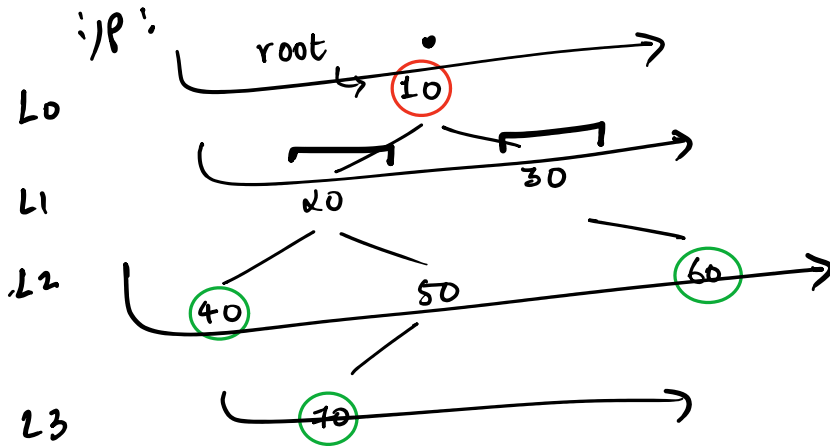
Distribute Coins in Binary Tree

Given the **root** of a binary tree with **N** nodes, where each node has **val** no. of coins such that the **total number** of coins across all the nodes in the tree is equal to **N**. Find the **minimum** no. of moves required to make each node have exactly one coin such that in a move you can choose any two adjacent nodes and move **one** coin from one node to another. In a move, a coin can go from a parent node to its child or from a child node to its parent.

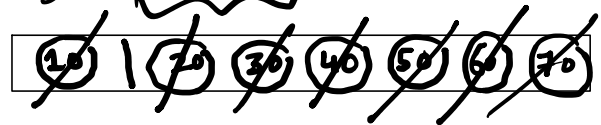


↳ Breadth first traversal of a Binary Tree

o/p: 10 20 30 40 50 60 70

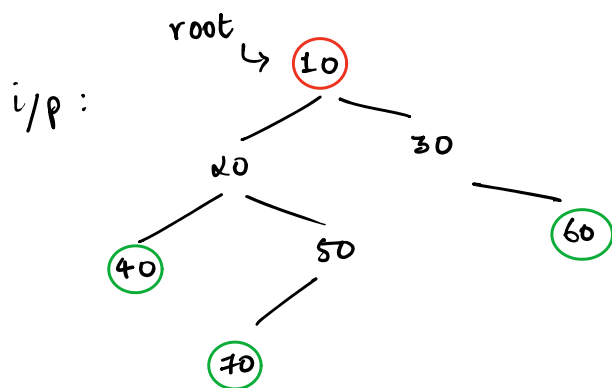


Queue, to keep track of nodes which have been visited but not yet processed.



o/p : 10 , 20 , 30 , 40 , 50 , 60 , 70

↳ Breadth first traversal of a Binary Tree

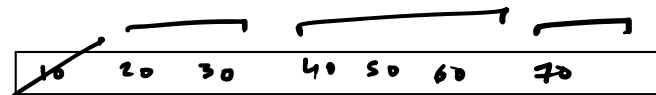
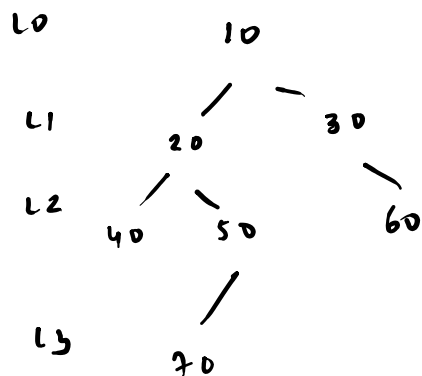


o/p :

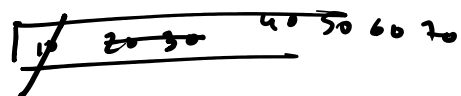
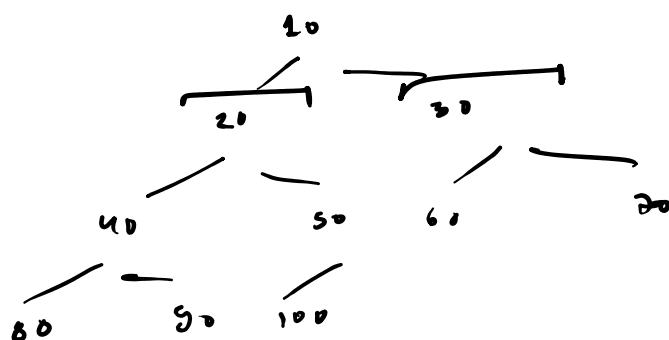
```

10
20 30
40 50 60
70
  
```

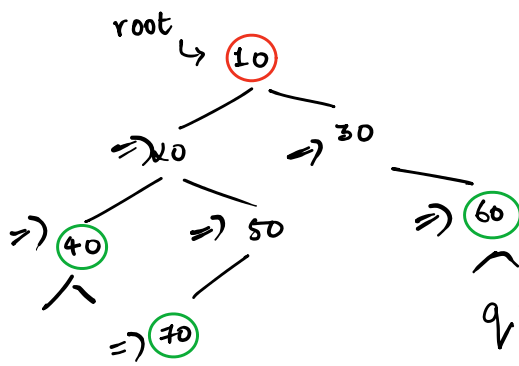
output is
formatted
level-wise



$\downarrow$ size = 1
 $\Rightarrow$ size = 2
 $\Rightarrow$ size = 3
 10 $\Rightarrow$ size = 1
 20 30
 40 50 60
 70



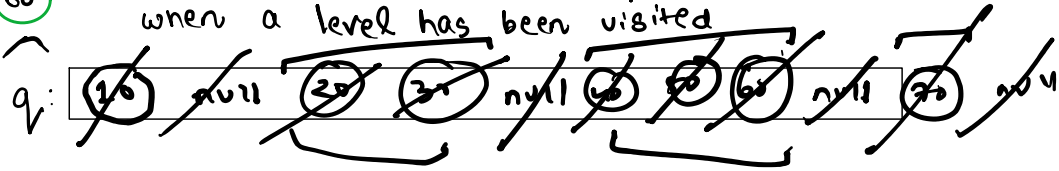
10
20 30



O/p :

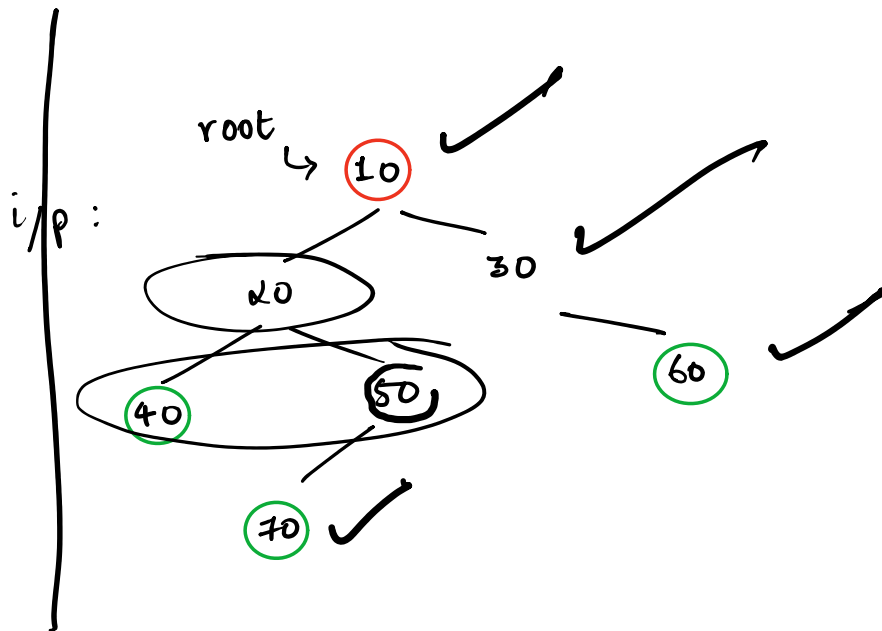
- **Queue**, to keep track of nodes which have been visited but not yet processed.

- Push **Null** value in the queue when a level has been visited

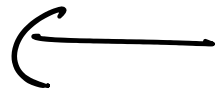


10 /n
 20 30 /n
 40 50 60 /n
 70 /n

[R][HW] Binary Tree Right View



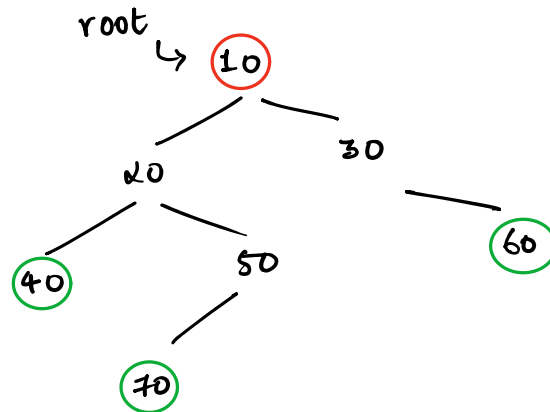
o/p : 10, 30, 60, 70



↳ Build a binary tree from level-order traversal

i/p - 10 20 30 40 50 -1 60 -1 -1 70 -1 -1 -1 -1 -1

o/p -



i/p - 10 20 30 40 50 -1 60 -1 -1 70 -1 -1 -1 -1 -1

Queue, to keep track of nodes whose child nodes are yet to be constructed.

q:

[HW] Build Tree from its Pre-Order and In-Order Traversal

pre + in $\rightarrow$ unique
post + in $\rightarrow$ BT

$\hookrightarrow$ Build a binary-tree from its pre-order and in-order traversals

i/p : pre - 10 20 40 50 70 30 60
in - 40 20 70 50 10 30 60

pre + post $\rightarrow$ not unique

o/p :

